OK, now that the services are running, we should be able to load our app in a browser. On your local machine, visit http://<instance IP address>/welcome, which for me is http://192.168.99.100/welcome (if you've forgotten the IP address of your instance, you can get it by running docker-machine ls). You should see our hit counter running correctly and incrementing on each page load. This shows that Redis is running and our app is connected to it successfully.

Similarly, visit http://<instance IP address>/users (for me, it's http://192.168.99.100/users), and you'll see our User scaffold showing that our database is running and our app is connected to it successfully.

We've now successfully deployed our application to the separate VirtualBox instance, which constitutes our single-node swarm cluster.

Tasks and Swarm's Scaling Model

Services running on Swarm are self-regulating. That is, we define a *desired state* for a service in terms of the number of containers that should run for it, and Swarm acts to ensure that this state is achieved and maintained.

It is this self-regulation that is key to how scaling is implemented, as well as Swarm's self-healing properties, which we'll see in the next chapter.

Swarm is made up of different parts. We can consider one part to be the *orchestrator*. Having told Swarm to deploy a service, the orchestrator is responsible for determining whether the service is in the desired state and, if not, taking action to correct this.

If the orchestrator sees that an additional container is needed, it creates a *task*, which represents the desire for a container to exist. It then sees what nodes are available in the cluster, and allocates the task to a node.

In our single-node cluster, the orchestrator has no choice but to allocate all of the tasks onto our only node. However, you can imagine that, in a multinode setup, tasks could be allocated evenly across multiple instances in the cluster.

Nodes in the cluster check with the orchestrator to know what tasks they've been allocated. They are then responsible for creating one container per task, thus achieving the desired state of the system.

An example should make this clearer. When we deployed our web service, we didn't specify the number of replica containers needed, so the default of one was assumed. The orchestrator sees that we want one web service container running, but that, initially, none are. It creates a task for the web service and